

# Project 2 Report

---

## 1. 实验目标与范围

本实验的目标是比较 C 与 Java 在不同数据类型和向量长度下的点乘性能，并分析编译优化级别对 C 版本的影响。我们关注以下维度：

- 语言：C 与 Java
- 数据类型：signed\_char、short、int、float、double
- 向量长度：N = {10, 100, 1000, 10000, 100000, 1000000}
- C 编译优化级：-O0、-O1、-O2、-O3

输出统一为 CSV，核心字段为：

lang,type,N, reps, total\_ns, avg\_ns, checksum

---

## 2.实验前的预期

1. 运行时间随着 N 增大基本线性增长：点乘是 O(N) 算法，N 增大时计算量线性增加，且数据访问模式简单，预期 avg\_ns 随 N 线性增长。
  2. 运行时间随着数据类型位宽增加而增长：更宽的数据类型（如 double）通常需要更多的计算资源和内存带宽，预期 avg\_ns 随数据类型位宽增加而增长。
  3. 性能随优化级提升：预计随着编译优化级别的提高，C 版本的性能将得到显著改善。
  4. 异常快/0ns/离散点：可能由于计时分辨率限制、优化级触发不同循环变换、系统噪声等原因导致某些点出现异常快或离散的结果。
  5. C 与 Java 性能对比预期：预计 C 版本在大多数情况下会比 Java 版本更快，特别是在开启-O3 优化级时。
- 

## 3. 实验环境

### 3.1 操作系统与内核

- OS: macOS 12.7.4
- 内核: Darwin 21.6.0 (x86\_64)

### 3.2 硬件环境

- CPU 型号: Intel(R) Core(TM) i5-1038NG7 CPU @ 2.00GHz
- 物理核心: 4
- 逻辑核心: 8
- 内存: 17179869184 bytes (约 16 GiB)

### 3.3 C 运行环境（编译/链接）

- cc --version: Apple clang 14.0.0 (clang-1400.0.29.202)
- clang --version: Apple clang 14.0.0 (clang-1400.0.29.202)

- Target: `x86_64-apple-darwin21.6.0`
- Thread model: `posix`

### 3.4 Java 运行环境 (JVM/JDK)

- `java -version`:
    - `openjdk version "17.0.17" 2025-10-21`
    - `OpenJDK Runtime Environment (build 17.0.17+10)`
    - `OpenJDK 64-Bit Server VM (build 17.0.17+10, mixed mode, sharing)`
  - `javac -version`: `javac 17.0.17`
- 

## 4. 软件程序

### 4.1 C 版本: `dotproduct.c`

关键实现点:

1. 使用 `xorshift64` 按固定 seed 生成输入数据, 保证可复现。
2. 对 5 种类型分别实现点乘函数:
  - `dot_schar_once`
  - `dot_short_once`
  - `dot_int_once`
  - `dot_float_once`
  - `dot_double_once`
3. `reps_for_n(n)` 根据 `N` 自动设定重复次数, 目标约 20M 次乘加, 范围 `[3, 2_000_000]`。
4. 计时使用 `clock_gettime(CLOCK_MONOTONIC)` (纳秒级, 单调时钟)。
5. 使用 `volatile` 全局 `checksum` 防止计算被优化掉。
6.
  - `__attribute__((noinline))` (禁止内联)
  - `asm volatile("" ::: "memory")` (编译器屏障) 目的是减少“循环被过度优化”带来的假快现象。

### 4.2 Java 版本: `Dotproduct.java`

关键实现点:

1. 与 C 保持同样的数据类型、同样的 `N` 集合、同样的 `repsForN` 策略。
  2. 计时使用 `System.nanoTime()`。
  3. 使用固定 seed 的 `Random` 填充输入, 保证可复现。
  4. 预热 (warmup) 阶段:
    - 分类型执行 10 轮预热, 先让 JIT 进入稳定阶段, 再正式计时。
  5. 同样通过 `volatile checksum` 保留可观察副作用, 降低死代码消除风险。
- 

## 5. 运行脚本与编译参数

### 5.1 C 脚本: `run_dotproduct_c.sh`

- `./run_dotproduct_c.sh 1`: 每个优化级输出 `results_c(0x).csv`

- `./run_dotproduct_c.sh 5`: 额外输出
  - `results_c(0x)_run1..run5.csv`
  - `results_c(0x)_mean.csv`
  - `results_c(0x)_median.csv`
- 编译参数: `-std=c11 -Wall -Wextra -O0/-O1/-O2/-O3`
- 编译选项介绍:
  - `-Wall -Wextra`: 开启所有警告, 帮助发现潜在问题。
  - `-O0/-O1/-O2/-O3`: 分别对应不同优化级别, 观察性能变化。
    - `-O0`: 无优化, 便于观察原始性能。
    - `-O1`: 基本优化, 开启常见的代码改进。
    - `-O2`: 更激进的优化, 开启更多的代码改进。
    - `-O3`: 最高级别的优化, 开启所有优化选项, 可能会改变代码结构以获得最大性能。

## 5.2 Java 脚本: `run_dotproduct_java.sh`

- `./run_dotproduct_java.sh 1`: 输出 `results_java.csv`
- `./run_dotproduct_java.sh 5`: 额外输出
  - `results_java_run1..run5.csv`
  - `results_java_mean.csv`
  - `results_java_median.csv`
- JVM 参数: `-Xms512m -Xmx512m -server`
- 说明: 固定堆大小用于减少堆扩容抖动; `-server` 在现代 JDK 下通常是默认模式, 但显式指定可读性更好。

---

## 6. 实验过程与关键结果

### 6.1 -O1 编译选项反常快于 -O2/-O3:

在最初的实验中, 发现 `-O1` 在 N 较小时 signed char, short, int 上表现异常快(平均时间为 0ns), 甚至比 `-O2` 和 `-O3` 更快。

```

projects > o1.csv > data
1 lang,type,N,reprs,total_ns,avg_ns,checksum
2 C,signed_char,10,2000000,1000,0.00,-250000000
3 C,signed_char,100,200000,0,0.00,-359200000
4 C,signed_char,1000,20000,1000,0.05,-345260000
5 C,signed_char,10000,2000,9000,4.50,-356770000
6 C,signed_char,100000,200,93000,465.00,-347166400
7 C,signed_char,1000000,20,836000,41800.00,-348140220
8 C,short,10,2000000,0,0.00,-923614140220
9 C,short,100,200000,0,0.00,-1401744340220
10 C,short,1000,20000,0,0.00,-870312820220
11 C,short,10000,2000,7000,3.50,-805490054220
12 C,short,100000,200,68000,340.00,-834381271620
13 C,short,1000000,20,2885000,144250.00,-848649640060
14 C,int,10,2000000,0,0.00,-1209262229305640060
15 C,int,100,200000,0,0.00,-1084038569558640060
16 C,int,1000,20000,1000,0.05,-139065931321940060
17 C,int,10000,2000,6000,3.00,-1457 Col 7: checksum
18 C,int,100000,200,126000,630.00,-1444455240116632260
19 C,int,1000000,20,766000,38300.00,-1439537793507279780
20 C,float,10,2000000,2828000,1.41,1519362.421706
21 C,float,100,200000,244000,1.22,1709330.855310
22 C,float,1000,20000,51000,2.55,2002245.513760
23 C,float,10000,2000,16000,8.00,2091637.019075
24 C,float,100000,200,131000,655.00,2109651.697204
25 C,float,1000000,20,1579000,78950.00,2102934.703357
26 C,double,10,2000000,2430000,1.22,1652184.836692
27 C,double,100,200000,243000,1.22,2393441.679082
28 C,double,1000,20000,26000,1.30,2349097.626149
29 C,double,10000,2000,14000,7.00,2392622.646148
30 C,double,100000,200,185000,925.00,2410297.528209
31 C,double,1000000,20,1764000,88200.00,2407731.074085
32

```

-O1, 未开启编译器屏障

```

projects > o3.csv > data
1 lang,type,N,reprs,total_ns,avg_ns,checksum
2 C,signed_char,10,2000000,222000,0.11,-250000000
3 C,signed_char,100,200000,22000,0.11,-359200000
4 C,signed_char,1000,20000,4000,0.20,-345260000
5 C,signed_char,10000,2000,11000,5.50,-356770000
6 C,signed_char,100000,200,84000,420.00,-347166400
7 C,signed_char,1000000,20,688000,34400.00,-348140220
8 C,short,10,2000000,126000,0.06,-923614140220
9 C,short,100,200000,13000,0.07,-1401744340220
10 C,short,1000,20000,2000,0.10,-870312820220
11 C,short,10000,2000,6000,3.00,-805490054220
12 C,short,100000,200,52000,260.00,-834381271620
13 C,short,1000000,20,505000,25250.00,-848649640060
14 C,int,10,2000000,92000,0.05,-1209262229305640060
15 C,int,100,200000,9000,0.04,-1084038569558640060
16 C,int,1000,20000,2000,0.10,-139065931321940060
17 C,int,10000,2000,5000,2.50,-1457346694859994060
18 C,int,100000,200,49000,245.00,-1444455240116632260
19 C,int,1000000,20,620000,31000.00,-1439537793507279780
20 C,float,10,2000000,2545000,1.27,1519362.421706
21 C,float,100,200000,244000,1.22,1709330.855310
22 C,float,1000,20000,25000,1.25,2002245.513760
23 C,float,10000,2000,14000,7.00,2091637.019075
24 C,float,100000,200,126000,630.00,2109651.697204
25 C,float,1000000,20,1383000,69150.00,2102934.703357
26 C,double,10,2000000,2429000,1.21,1652184.836692
27 C,double,100,200000,243000,1.22,2393441.679082
28 C,double,1000,20000,25000,1.25,2349097.626149
29 C,double,10000,2000,15000,7.50,2392622.646148
30 C,double,100000,200,143000,715.00,2410297.528209
31 C,double,1000000,20,1506000,75300.00,2407731.074085
32

```

-O3, 未开启编译器屏障

为了理解 `-O1` 反常现象，我们深入探讨各编译优化级别的特点及其对点乘性能的影响。

### -O0：无优化（调试模式）

主要特性：

- 每个变量都从内存读写，不缓存到寄存器
- 循环结构完全保留，不展开、不合并
- 函数调用完全保留，不内联

- 编译速度最快，生成的二进制可直接用 GDB 调试，变量值与源码一一对应

**适用场景：** 调试阶段使用，性能最差，是其他优化级别的性能基准参照。

### -01：基本优化

在不显著增加编译时间的前提下，开启一批收益明确的优化。

**主要优化手段：**

- **常量折叠：**  $x = 2 + 3$  直接替换为  $x = 5$ ，无需运行时计算
- **死代码消除（DCE）：** 删除结果从未被使用的计算语句
- **基本内联：** 对极短小的函数进行内联展开，消除函数调用开销
- **寄存器分配优化：** 将频繁访问的变量缓存到寄存器，减少不必要的内存读写

### -02：激进优化（生产环境常用默认）

在 -01 基础上增加更多分析和变换，适合大多数生产环境。

**主要优化手段：**

- **循环不变量外提（LICM）：** 将循环内不随迭代变化的计算移到循环外，避免重复执行
- **公共子表达式消除（CSE）：** 相同的子表达式只计算一次，结果复用
- **函数内联扩展：** 更积极地将函数体展开到调用处，减少调用开销
- **指令调度：** 重排指令顺序，减少 CPU 流水线停顿，提高指令吞吐量
- **分支预测优化：** 调整代码布局，配合 CPU 的分支预测器减少预测失败惩罚

**特点：** 不会改变浮点运算顺序，计算结果与 -00 在数值上保持一致。

### -03：最高优化

在 -02 基础上开启可能改变代码结构的激进变换。

**主要优化手段：**

- **自动向量化（Auto-vectorization）：** 将标量循环转换为 SIMD 指令（SSE/AVX），一条指令同时处理多个元素，是点积计算性能提升的核心来源
- **循环展开（Loop unrolling）：** 减少循环控制指令的开销，同时提高指令级并行度
- **函数克隆（Function cloning）：** 针对不同调用场景生成特化版本，进一步优化热路径
- **更激进的内联：** 几乎所有 `inline` 标记的函数都会被展开

**向量化示意：**

标量循环（00/01/02）：

```
for i in 0..n:
    acc += a[i] * b[i]
```

向量化后（03，概念示意）：

```
for i in 0..n step 4:
    acc_vec += a[i:i+4] * b[i:i+4]
acc = sum(acc_vec)
```

对于 `double` 类型，AVX2 指令集可一次处理 4 个元素；`float` 类型可一次处理 8 个元素，因此浮点类型在 `-O3` 下的加速效果尤为显著。

同时我让大模型帮我画了一张图总结：

| 优化特性          | -O0 | -O1    | -O2  | -O3  |
|---------------|-----|--------|------|------|
| 常量折叠          | —   | ✓      | ✓    | ✓    |
| 死代码消除         | —   | ✓      | ✓    | ✓    |
| 基本内联          | —   | ✓      | ✓    | ✓    |
| 循环不变量外提       | —   | —      | ✓    | ✓    |
| 指令调度          | —   | —      | ✓    | ✓    |
| 自动向量化         | —   | —      | —    | ✓    |
| 循环展开          | —   | —      | —    | ✓    |
| 可能改变代码结构      | 否   | 否      | 否    | 是    |
| 典型加速比（vs -O0） | 1×  | 1.5–2× | 2–4× | 3–6× |

因此我分析原因可能是：

- 对于 `-O1`，DCE 过于激进时会把"看似无意义"的循环整体删除。例如在基准测试中，若点积函数被声明为 `static inline` 且每次调用的输入完全不变，编译器可能将整个重复循环替换为一次计算加一次乘法，导致计时结果异常偏低（接近 0ns）；
- 而在 `-O2` 和 `-O3` 中，虽然优化更激进，但由于引入了更多的变换（如向量化、循环展开等），反而可能保留了更多的计算步骤，使得结果更接近实际的计算时间。

所以我采取了 `asm volatile("" ::: "memory")`（编译器屏障）来强制编译器保留计算过程，避免过度优化导致的假快现象。这段代码的作用是告诉编译器“这里有一个内存访问，可能会影响程序状态”，从而阻止编译器将相关代码优化掉或重排。得到的结果如下：

```

projects > o1.csv > data
1  lang,type,N,reprs,total_ns,avg_ns,checksum
2  C,signed_char,10,2000000,15581000,7.79,-250000000
3  C,signed_char,100,200000,14143000,70.72,-359200000
4  C,signed_char,1000,20000,12070000,603.50,-345260000
5  C,signed_char,10000,2000,12083000,6041.50,-356770000
6  C,signed_char,100000,200,12481000,62405.00,-347166400
7  C,signed_char,1000000,20,12886000,644300.00,-348140220
8  C,short,10,2000000,10976000,5.49,-923614140220
9  C,short,100,200000,13821000,69.11,-1401744340220
10 C,short,1000,20000,12286000,614.30,-870312820220
11 C,short,10000,2000,12066000,6033.00,-805490054220
12 C,short,100000,200,13224000,66120.00,-834381271620
13 C,short,1000000,20,16756000,837800.00,-848649640060
14 C,int,10,2000000,10897000,5.45,-1209262229305640060
15 C,int,100,200000,13060000,65.30,-1084038569558640060
16 C,int,1000,20000,12132000,606.60,-1390965931321940060
17 C,int,10000,2000,13664000,6832.00,-1457346694859994060
18 C,int,100000,200,13124000,65620.00,-1444455240116632260
19 C,int,1000000,20,15052000,752600.00,-1439537793507279780
20 C,float,10,2000000,16383000,8.19,1519362.421706
21 C,float,100,200000,22319000,111.59,1709330.855310
22 C,float,1000,20000,24707000,1235.35,2002245.513760
23 C,float,10000,2000,25414000,12707.00,2091637.019075
24 C,float,100000,200,26823000,134115.00,2109651.697204
25 C,float,1000000,20,28943000,1447150.00,2102934.703357
26 C,double,10,2000000,10713000,5.36,1652184.836692
27 C,double,100,200000,16138000,80.69,2393441.679082
28 C,double,1000,20000,24121000,1206.05,2349097.626149
29 C,double,10000,2000,25839000,12919.50,2392622.646148
30 C,double,100000,200,27682000,138410.00,2410297.528209
31 C,double,1000000,20,29313000,1465650.00,2407731.074085
32

```

-O1, 开启编译器屏障

和前面的结果比较，这样就明显改善了 `-O1` 在小 N 时的异常快现象，结果更符合预期。

## 6.2

# 6. 现象解释

## 5.1 为什么性能随 N 增大基本线性增长

点乘是典型  $O(N)$  顺序访问。N 增大时：

- 计算次数线性增加
- 数据工作集逐渐超出 L1/L2 cache
- 延迟更多转为内存带宽主导

因此 `avg_ns` 大体随 N 线性增长。

## 5.2 为什么 `float` 不一定比 `double` 更快

尽管理论上 `float` 更省带宽，但真实结果受以下因素共同影响：

- 编译器/JIT 的向量化策略
- 指令混合与寄存器分配
- 累加精度（本实验中 float 计算累加到 double）

因此不同语言/实现中，`float` 与 `double` 的相对关系可能变化。

## 5.3 为什么会出现异常快/0ns/离散点

常见原因：

- 计时窗口过短导致分辨率量化误差

- 优化级触发不同循环变换
- 运行期系统噪声（调度、频率、后台任务）

当前版本已加入 `noinline + memory barrier + 多轮统计(mean/median)`，显著改善了稳定性，但完全消除噪声不现实。

---

## 7. 有效性与局限

1. 硬件/系统信息未在 **CSV** 自动记录：不同机器不可直接横比。
2. 单机单时段测量：受温度、负载、节能策略影响。
3. 仅测标量实现：未引入显式 SIMD intrinsic / Java Vector API。
4. 随机分布固定：有利可复现，但不覆盖所有数据分布场景。

---

## 8. 可复现实验步骤

在 `projects` 目录执行：

```
# C: 四种优化级, 每级5次, 输出 run/mean/median
./run_dotproduct_opt.sh 5

# Java: 5次, 输出 run/mean/median
./run_dotproduct_java.sh 5
```

核心结果文件：

- C: `results_c(00~03)_mean.csv`、`results_c(00~03)_median.csv`
- Java: `results_java_mean.csv`、`results_java_median.csv`

---

## 9. 结论（当前数据下）

1. C 优化级对性能影响显著，`-03` 相对 `-00` 的提升普遍在 `2x~5x`。
2. `-01` 在浮点上与 `-03` 可接近，但在整型上通常明显慢于 `-03`。
3. Java 在 `double` 上与 C(`-03`) 已非常接近；在 `float/int` 上整体仍偏慢。
4. 使用 `median` 比单次结果更可信，能更好反映“典型性能”。

---

## 10. 你后续可直接改的地方（建议）

- 补充你的真实硬件信息（CPU 型号、核心数、内存、JDK 版本）。
- 从 `*_median.csv` 里摘 2~3 张关键图（N-log 坐标更清晰）。
- 增加“每元素耗时 (avg\_ns / N)”表，便于跨 N 对比。
- 如果需要课程讨论，可补充 `-Ofast`、`-march=native` 与 Java Vector API 的扩展实验。